FIELD NOTES / AI, EVIDENCE & JUDGMENT

The AI Paradoxes

A reflection on building a CV-verification agent

A model can sound certain while choosing the wrong person. It can obey a document when it should treat that document as evidence. And a single score can look precise while hiding uncertainty, tool failures, and judgment calls. Building this verifier made those tensions concrete.

1. Fluent reasoning is not reliable identification

The clearest example was a CV for Daniel Wilson, a name shared by several LinkedIn profiles in Munich. One profile, ID 7440, matched the city, legal headline, and skills. Another, ID 6561, matched the distinctive evidence: Tencent and Carousell roles, their dates and junior levels, and a PolyU MBA in Legal. In one run, the model selected 7440 and confidently called the CV fabricated; in another, it selected 6561 and scored the same CV highly.

This is a paradox of fluent intelligence: the explanation can be coherent relative to the wrong premise. More searching did not automatically solve the problem, because the model inspected candidates inconsistently and over-weighted easy, generic matches. I added a local candidate-ranking tool that retrieves profiles and scores overlap in employers, titles, dates, education, and skills before the model judges discrepancies. That made the evidence path more repeatable, but the heuristic is still not identity proof: text matching can miss aliases, formatting differences, or meaningful contradictions. Reliability comes from the whole system, not from how persuasive the final paragraph sounds.

2. Instruction following meets untrusted input

The Task 2 exercise exposed another tension. A PDF is supposed to provide evidence about a candidate, yet its extracted text is placed into the same model conversation as ordinary user content. Tiny white text can therefore act as an indirect prompt injection: it can imitate an evaluator note and ask the model to ignore certain claims. The text has no real system-level authority, but the model may still treat its wording as guidance.

This is not simply a contest between a clever prompt and a clever model. It is a boundary-design problem. The verifier should label document contents as untrusted data, isolate instructions from evidence, and apply a fixed comparison procedure regardless of what the CV says about how it should be scored. Red-team testing is valuable because it reveals that the model’s ability to follow instructions is also a vulnerability when it cannot reliably distinguish whose instructions deserve authority.

3. A precise score can conceal imprecision

The program expects one number, but the model sometimes returns a detailed explanation containing profile IDs, dates, and then a score. A strict parser that searches for exactly one number rejects that answer. At other times, a network exception is caught and converted to 0.0. Both outcomes can look like a low-confidence judgment in a CSV, although one is a formatting failure and the other is no judgment at all.

This is the paradox of measurement: converting a complex decision into a scalar makes results easy to compare, but can erase why the result occurred. A score should be accompanied internally by a structured audit record: selected profile, field-by-field matches, confirmed discrepancies, unresolved evidence, and execution status. Even if the assignment requires a single number as output, the system should distinguish “discrepancy found” from “could not verify.” A failure to reach the server should never be mistaken for evidence that a CV is false.

4. Rules improve consistency, but rules have blind spots

The verifier checks a defined set of fields: identity, city, employment, education, and skills. It treats job descriptions as out of scope. That makes grading more consistent and protects candidates from speculative judgments about prose, but it also means a fabricated achievement inside a job description can pass unnoticed. The attack succeeded locally because the structured facts remained consistent while the false claim sat outside the required coverage.

This is a paradox of fairness and completeness. A narrow rubric is easier to apply consistently; a broad rubric can catch more falsehoods but invites unsupported inference. The responsible answer is not to let the model judge everything. It is to make coverage explicit, require evidence for every checked claim, and state clearly when a claim is outside scope. In a real verification setting, consequential achievement claims would need a suitable source or a human review, not a guess based on plausibility.

5. Automation shifts work rather than removing it

The agent can search profiles and compare many fields quickly, but its speed moves effort into new places: deciding which candidate is the right one, checking whether the tools returned complete data, handling network and proxy failures, and reviewing edge cases where language does not map neatly to profile fields. Automation does not eliminate judgment; it concentrates judgment at system boundaries and in exceptions.

My main takeaway is that an AI verifier should be treated as an evidence-organizing assistant, not an oracle. Its conclusions are strongest when identity is well resolved, fields are explicitly scoped, tool results are traceable, and uncertainty remains visible. The most important paradox is that capability and fragility grow together: the more authority we give a model to act on evidence, the more carefully we must design the boundaries that keep evidence, instructions, and confidence separate.

Reflection grounded in the development and testing of the FTEC5660 CV-verification agent.